

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE			DEPARTMENT OF COMPUTER SCIENCE ENGINEERING		
Program Name: B. Tech		Assignment Type: Lab		Academic Year:2025-2026	
Course Coordinator Name		Dr. Rishabh Mittal			
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju			
Year/Sem	III/II	Regulation	R23		
Date and Day of Assignment	Week3 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52		
Duration	2 Hours	Applicable to Batches	All batches		
AssignmentNumber:6.3(Present assignment number)/24(Total number of assignments)					
Q.No.	Question			Expected Time to complete	
1	Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals Lab Objectives • To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements. • To analyze how AI tools suggest logic for object-oriented programming and control structures. • To evaluate the correctness, readability, and completeness of AI-generated Python code.			Week3 - Wednesday	

Lab Outcomes (LOs)

After completing this lab, students will be able to:

- Use AI tools to generate and complete Python class definitions and methods.
- Understand and assess AI-suggested loop constructs for iterative tasks.
- Generate and evaluate conditional statements using AI-driven prompts.
- Critically analyze AI-assisted code for correctness, clarity, and efficiency.

Task Description #1: Classes (Student Class)

Scenario

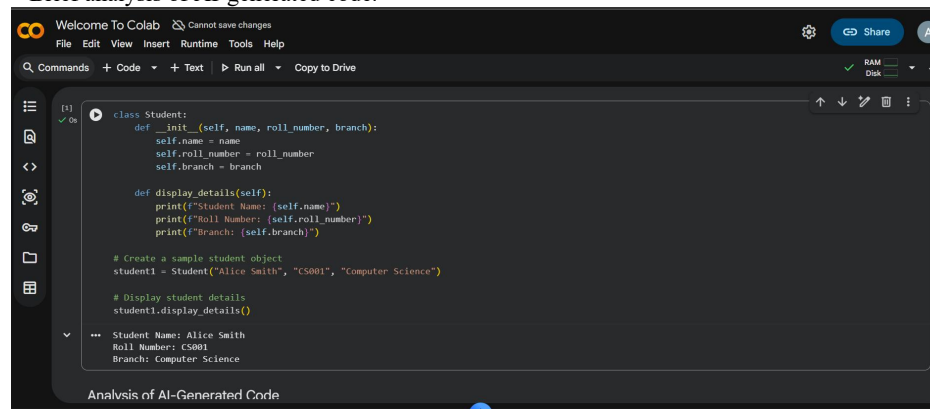
You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method `display_details()` to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

- A Python class with a constructor (`__init__`) and a `display_details()` method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code.



The screenshot shows a Google Colab notebook interface. The code cell contains the following Python code:

```
class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print(f"Student Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Branch: {self.branch}")

# Create a sample student object
student1 = Student("Alice Smith", "CS001", "Computer Science")

# Display student details
student1.display_details()
```

The output cell shows the following text:

```
Student Name: Alice Smith
Roll Number: CS001
Branch: Computer Science
```

Below the output, there is a section titled "Analysis of AI-Generated Code" which is currently collapsed.

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

[2]
✓ Os

```
def print_multiples_for_loop(number):  
    """  
    Prints the first 10 multiples of a given number using a for loop.  
    """  
    print(f"First 10 multiples of {number} (using for loop):")  
    for i in range(1, 11): # Loop 10 times, from 1 to 10  
        print(number * i)  
  
    # Example usage:  
    print_multiples_for_loop(5)
```



```
First 10 multiples of 5 (using for loop):  
5  
10  
15  
20  
25  
30  
35  
40  
45  
50
```

[3]
✓ Os



```
def print_multiples_while_loop(number):  
    """  
    Prints the first 10 multiples of a given number using a while loop.  
    """  
    print(f"\nFirst 10 multiples of {number} (using while loop):")  
    count = 1 # Initialize a counter  
    while count <= 10: # Loop as long as the counter is less than or equal to 10  
        print(number * count)  
        count += 1 # Increment the counter  
  
    # Example usage:  
    print_multiples_while_loop(7)
```



```
...  
First 10 multiples of 7 (using while loop):  
7  
14  
21  
28  
35  
42  
49  
56  
63
```

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

```

▶ def classify_age_if_elif_else(age):
    """
    Classifies an age into categories using if-elif-else statements.
    """
    if age < 13:
        return "Child"
    elif age < 18:
        return "Teenager"
    elif age < 65:
        return "Adult"
    else:
        return "Senior"

    # Test cases
    print(f"Age 10: {classify_age_if_elif_else(10)}")
    print(f"Age 15: {classify_age_if_elif_else(15)}")
    print(f"Age 30: {classify_age_if_elif_else(30)}")
    print(f"Age 70: {classify_age_if_elif_else(70)}")

```

```

... Age 10: Child
    Age 15: Teenager
    Age 30: Adult
    Age 70: Senior

```

```

▶ def classify_age_simplified(age):
    """
    Classifies an age into categories using simplified conditions (list of tuples).
    """
    age_groups = [
        (13, "Child"),
        (18, "Teenager"),
        (65, "Adult")
    ]

    for limit, group_name in age_groups:
        if age < limit:
            return group_name
    return "Senior" # Default for ages 65 and above

    # Test cases
    print(f"\nAge 10 (simplified): {classify_age_simplified(10)}")
    print(f"Age 15 (simplified): {classify_age_simplified(15)}")
    print(f"Age 30 (simplified): {classify_age_simplified(30)}")
    print(f"Age 70 (simplified): {classify_age_simplified(70)}")

```

```

... Age 10 (simplified): Child
    Age 15 (simplified): Teenager
    Age 30 (simplified): Adult
    Age 70 (simplified): Senior

```

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.

- Explanation and comparison of different approaches.

```
def sum_to_n_for_loop(n):
    """
    Calculates the sum of the first n natural numbers using a for loop.
    """
    total_sum = 0
    for i in range(1, n + 1):
        total_sum += i
    return total_sum

# Test cases
print(f"Sum of first 5 numbers (for loop): {sum_to_n_for_loop(5)}")
print(f"Sum of first 10 numbers (for loop): {sum_to_n_for_loop(10)}")

Sum of first 5 numbers (for loop): 15
Sum of first 10 numbers (for loop): 55
```

Analysis of `sum_to_n_for_loop`

Correctness: This `for` loop implementation correctly calculates the sum of the first 'n' natural numbers. It initializes `total_sum` to 0 and then iterates through each number from 1 to `n` (inclusive), adding each to the `total_sum`.

Clarity: The code is clear and easy to understand. The variable names (`n`, `total_sum`, `i`) are descriptive, and the logic follows a straightforward iterative summation process.

```
[7]
✓ 0s def sum_to_n_formula(n):
    """
    Calculates the sum of the first n natural numbers using the mathematical formula n*(n+1)/2.
    """
    if n < 0:
        return "Input must be a non-negative number"
    return n * (n + 1) // 2 # Using integer division

# Test cases
print(f"Sum of first 5 numbers (formula): {sum_to_n_formula(5)}")
print(f"Sum of first 10 numbers (formula): {sum_to_n_formula(10)}")
print(f"Sum of first 0 numbers (formula): {sum_to_n_formula(0)}")

Sum of first 5 numbers (formula): 15
Sum of first 10 numbers (formula): 55
Sum of first 0 numbers (formula): 0
```

Analysis of `sum_to_n_formula` (Mathematical Formula)

Correctness: This implementation uses the well-known mathematical formula $n * (n + 1) / 2$ to directly calculate the sum of the first 'n' natural numbers. This formula is derived from arithmetic progression and is mathematically sound.

Clarity: The code is very concise due to the direct application of the formula. It's immediately clear what the function aims to do if one is

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as `deposit()`, `withdraw()`, and `check_balance()`.
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation.

#code

```
class BankAccount:
    def __init__(self, account_holder, initial_balance=0):
        """
        Initializes a new BankAccount object.

        Args:
            account_holder (str): The name of the account holder.
            initial_balance (float): The starting balance of the account.
        Defaults to 0.
        """
        self.account_holder = account_holder
```

```

        self.balance = initial_balance
        print(f"Account created for {self.account_holder} with initial balance:
        ${self.balance:.2f}")

    def deposit(self, amount):
        """
        Deposits a specified amount into the account.

        Args:
            amount (float): The amount to deposit.
        """
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount:.2f}. New balance: ${self.balance:.2f}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        """
        Withdraws a specified amount from the account.

        Args:
            amount (float): The amount to withdraw.
        """
        if amount <= 0:
            print("Withdrawal amount must be positive.")
        elif self.balance >= amount:
            self.balance -= amount
            print(f"Withdrew: ${amount:.2f}. New balance: ${self.balance:.2f}")
        else:
            print(f"Insufficient funds. Current balance: ${self.balance:.2f}")

    def check_balance(self):
        """
        Prints the current balance of the account.
        """
        print(f"Current balance for {self.account_holder}: ${self.balance:.2f}")

# --- Demonstration of BankAccount operations ---

# Create a bank account
my_account = BankAccount("John Doe", 1000.00)

# Check initial balance
my_account.check_balance()

# Deposit funds
my_account.deposit(500.00)
my_account.deposit(-100.00) # Invalid deposit

# Withdraw funds
my_account.withdraw(200.00)
my_account.withdraw(1500.00) # Insufficient funds
my_account.withdraw(-50.00) # Invalid withdrawal

# Check final balance

```

```
my_account.check_balance()
```

OUTPUT:

Account created for John Doe with initial balance: \$1000.00 Current balance for John Doe: \$1000.00 Deposited: \$500.00. New balance: \$1500.00 Deposit amount must be positive. Withdrew: \$200.00. New balance: \$1300.00 Insufficient funds. Current balance: \$1300.00 Withdrawal amount must be positive. Current balance for John Doe: \$1300.00

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.